

DATA STRUCTURES - IT23302 HW ASSIGNMENT SET - 4

Name : V Rahul

Roll Number : 2025115120

Batch : IT Batch II Second Year

Date : 06/09/2026

1. Implement a stack using Queue data structure.

```
#include<stdio.h>
```

```
#define MAX 100
```

```
int queue[MAX];
```

```
int front=0,rear=-1;
```

```
void enqueue(int x)
```

```
{
```

```
    queue[++rear]=x;
```

```
}
```

```
int dequeue()
```

```
{
```

```
    return queue[front++];
```

```
}
```

```
void push(int x)
```

```
{
```

```
    int n=rear-front+1;
```

```
int i;
```

```
enqueue(x);
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    enqueue(dequeue());
```

```
}
```

```
}
```

```
int pop()
```

```
{
```

```
    if(front>rear)
```

```
    {
```

```
        printf("Stack underflow\n");
```

```
        return -1;
```

```
    }
```

```
    return dequeue();
```

```
}
```

```
void display()
```

```
{
```

```
    int i;
```

```
    if(front>rear)
```

```
    {
```

```
    printf("Stack is empty\n");  
    return;  
}
```

```
printf("Stack: ");
```

```
for(i=front;i<=rear;i++)  
{  
    printf("%d ",queue[i]);  
}
```

```
printf("\n");  
}
```

```
int main()  
{  
    push(10);  
    push(20);  
  
    display();  
  
    printf("Popped element: %d\n",pop());  
  
    display();  
  
    return 0;  
}
```

OUTPUT

PS C:\My Files\01_ACADEMICS\SEMESTER_3\DSA\hw_4> ./1

Stack: 20 10

Popped element: 20

Stack: 10

=== Code Execution Successful ===

2. Implement a Queue using Stack data structure.

```
#include<stdio.h>
```

```
#define MAX 100
```

```
int s1[MAX],s2[MAX];
```

```
int top1=-1,top2=-1;
```

```
void push1(int x)
```

```
{
```

```
    s1[++top1]=x;
```

```
}
```

```
int pop1()
```

```
{
```

```
    return s1[top1--];
```

```
}
```

```
void push2(int x)
```

```
{  
    s2[++top2]=x;  
}
```

```
int pop2()
```

```
{  
    return s2[top2--];  
}
```

```
void enqueue(int x)
```

```
{  
    push1(x);  
}
```

```
int dequeue()
```

```
{  
    if(top1== -1 && top2== -1)  
    {  
        printf("Queue underflow\n");  
        return -1;  
    }
```

```
    if(top2== -1)
```

```
{  
    while(top1!= -1)
```

```

        {
            push2(pop1());
        }
    }

    return pop2();
}

void display()
{
    int i;

    if(top1== -1 && top2== -1)
    {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue: ");

    for(i=top2;i>=0;i--)
    {
        printf("%d ",s2[i]);
    }

    for(i=0;i<=top1;i++)
    {

```

```
        printf("%d ",s1[i]);
    }

    printf("\n");
}

int main()
{
    enqueue(10);
    enqueue(20);

    display();

    printf("Dequeued: %d\n",dequeue());

    display();

    return 0;
}
```

OUTPUT

PS C:\My Files\01_ACADEMICS\SEMESTER_3\DSA\hw_4> ./2

Queue: 10 20

Dequeued: 10

Queue: 20

=== Code Execution Successful ===

3. Write a C program using stack data structure to implement managing the history of web pages visited.

```
#include<stdio.h>
```

```
#include<string.h>
```

```
#define MAX 100
```

```
char history[MAX][100];
```

```
int top=-1;
```

```
void visit(char page[])
```

```
{
```

```
    if(top==MAX-1)
```

```
    {
```

```
        printf("History is full\n");
```

```
        return;
```

```
    }
```

```
    top++;
```

```
    strcpy(history[top],page);
```

```
    printf("Visited: %s\n",page);
```

```
}
```



```
void back()
{
    if(top<=0)
    {
        printf("No previous page\n");
        return;
    }

    printf("Going back from: %s\n",history[top]);

    top--;

    printf("Current page: %s\n",history[top]);
}

void display()
{
    int i;

    if(top== -1)
    {
        printf("History is Empty\n");
        return;
    }

    printf("Web page History:\n");
```

```
    for(i=top;i>=0;i--)
    {
        printf("%s\n",history[i]);
    }
}
```

```
int main()
{
    visit("Google");
    visit("Youtube");

    display();

    back();

    return 0;
}
```

OUTPUT

PS C:\My Files\01_ACADEMICS\SEMESTER_3\DSA\hw_4> ./3

Visited: Google

Visited: Youtube

Web page History:

Youtube

Google

Going back from: Youtube

Current page: Google

=== Code Execution Successful ===

4. Write a C program to sort a stack using temporary stack.

```
#include<stdio.h>
```

```
#define MAX 100
```

```
int stack[MAX],temp[MAX];
```

```
int top=-1,ttop=-1;
```

```
void push(int x)
```

```
{
```

```
    stack[++top]=x;
```

```
}
```

```
int pop()
```

```
{
```

```
    return stack[top--];
```

```
}
```

```
void tpush(int x)
```

```
{
```

```
    temp[++ttop]=x;
```

```
}
```

```
int tpop()
{
    return temp[ttop--];
}
```

```
void sort_stack()
{
    int x;

    while(top!=-1)
    {
        x=pop();

        while(ttop!=-1 && temp[ttop]>x)
        {
            push(tpop());
        }

        tpush(x);
    }

    while(ttop!=-1)
    {
        push(tpop());
    }
}
```

```
void display()
{
    int i;

    printf("Stack: ");

    for(i=top;i>=0;i--)
    {
        printf("%d ",stack[i]);
    }

    printf("\n");
}
```

```
int main()
{
    int arr[5]={3,1,5,2,4};

    int i;

    for(i=0;i<5;i++)
    {
        push(arr[i]);
    }

    printf("OG ");
    display();
}
```

```
sort_stack();

printf("Sorted ");
display();

return 0;
}
```

OUTPUT

PS C:\My Files\01_ACADEMICS\SEMESTER_3\DSA\hw_4> ./4

OG Stack: 4 2 5 1 3

Sorted Stack: 5 4 3 2 1

=== Code Execution Successful ===

5. The Stock Span Problem is a financial problem where we have a series of daily price changes for a stock denoted by an array. The task is to calculate the span of the stock's price for all days. The span of a stock's price for i th day gives the maximum number of consecutive leading up to i th day, where stock's price is less than or equal to the price on day i .

```
#include<stdio.h>
```

```
#define MAX 100
```

```
int main()
```

```
{
```

```
    int price[MAX],span[MAX];
```

```
int stack[MAX];
```

```
int top=-1,n,i;
```

```
printf("Enter no. of days: ");
```

```
scanf("%d",&n);
```

```
printf("Enter stock prices: ");
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    scanf("%d",&price[i]);
```

```
}
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    while(top!=-1 && price[stack[top]]<=price[i])
```

```
    {
```

```
        top--;
```

```
    }
```

```
    if(top==-1)
```

```
    {
```

```
        span[i]=i+1;
```

```
    }
```

```
    else
```

```

    {
        span[i]=i-stack[top];
    }

    stack[++top]=i;
}

printf("Stock span: ");

for(i=0;i<n;i++)
{
    printf("%d ",span[i]);
}

printf("\n");

return 0;
}

```

OUTPUT

PS C:\My Files\01_ACADEMICS\SEMESTER_3\DSA\hw_4> ./5

Enter no. of days: 7

Enter stock prices: 100 80 60 70 60 75 85

Stock span: 1 1 1 2 1 4 6

=== Code Execution Successful ===

6. Given an array, for each element find the nearest element to its right that has a higher frequency than the current element. If no such element exists, set the value to -1. Implement using Stack.

```
#include<stdio.h>
```

```
#define MAX 100
```

```
int main()
```

```
{
```

```
    int arr[MAX],freq[MAX],ans[MAX],stack[MAX];
```

```
    int n,i,j,top=-1;
```

```
    printf("Enter the size of array : ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter array elements : ");
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        freq[i]=0;
```

```
        for(j=0;j<n;j++)
```

```
{  
    if(arr[i]==arr[j])  
    {  
        freq[i]++;  
    }  
}  
}
```

```
for(i=n-1;i>=0;i--)  
{  
    while(top!=-1 && freq[stack[top]]<=freq[i])  
    {  
        top--;  
    }
```

```
    if(top== -1)  
    {  
        ans[i]=-1;  
    }  
    else  
    {  
        ans[i]=arr[stack[top]];  
    }
```

```
    stack[++top]=i;  
}
```

```

printf("Output: ");

for(i=0;i<n;i++)
{
    printf("%d ",ans[i]);
}

printf("\n");

return 0;
}

```

OUTPUT

PS C:\My Files\01_ACADEMICS\SEMESTER_3\DSA\hw_4> ./6

Enter the size of array : 6

Enter array elements : 2 1 1 3 2 1

Output: 1 -1 -1 2 1 -1

=== Code Execution Successful ===

7. Given a pattern containing only I's and D's. I for increasing and D for decreasing. Devise an algorithm using stack to print the minimum number following that pattern. Digits from 1-9 and digits can't repeat.

```
#include<stdio.h>
```

```
#define MAX 100
```

```
int stack[MAX];
```

```
int top=-1;
```

```
void push(int x)
```

```
{  
    stack[++top]=x;  
}
```

```
int pop()
```

```
{  
    return stack[top--];  
}
```

```
int main()
```

```
{  
    char pattern[MAX];
```

```
    int i=0,num=1;
```

```
    printf("Enter pattern : ");
```

```
    scanf("%s",pattern);
```

```
    while(pattern[i]!='\0')
```

```
{  
    push(num);  
    num++;
```

```

        if(pattern[i]!='I')
        {
            while(top!=-1)
            {
                printf("%d",pop());
            }
        }

        i++;
    }

    push(num);

    while(top!=-1)
    {
        printf("%d",pop());
    }

    printf("\n");

    return 0;
}

```

OUTPUT

PS C:\My Files\01_ACADEMICS\SEMESTER_3\DSA\hw_4> ./7

Enter pattern : IDID

13254

=== Code Execution Successful ===

8. Given an array of digits (values are from 0 to 9), find the minimum possible sum of two numbers formed from digits of the array. All digits of the given array must be used to form the two numbers.

```
#include<stdio.h>
```

```
#define MAX 100
```

```
int main()
```

```
{
```

```
    int arr[MAX];
```

```
    int n,i,j,temp;
```

```
    int num1=0,num2=0;
```

```
    printf("Enter the size of array : ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter array elements : ");
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        scanf("%d",&arr[i]);
```

```
    }
```

```
for(i=0;i<n;i++)
{
    for(j=0;j<n-i-1;j++)
    {
        if(arr[j]>arr[j+1])
        {
            temp=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=temp;
        }
    }
}
```

```
for(i=0;i<n;i++)
{
    if(i%2==0)
    {
        num1=num1*10+arr[i];
    }
    else
    {
        num2=num2*10+arr[i];
    }
}
```

```
printf("Minimum Sum : %d\n",num1+num2);
```

```
    return 0;
}
```

OUTPUT

PS C:\My Files\01_ACADEMICS\SEMESTER_3\DSA\hw_4> ./8

Enter the size of array : 6

Enter array elements : 6 8 4 5 2 3

Minimum Sum : 604

=== Code Execution Successful ===

9. Given an array and a positive integer K, find the first negative integer for each window (contiguous subarray) of size K. If a window does not contain a negative integer, print 0.

```
#include<stdio.h>
```

```
#define MAX 100
```

```
int main()
```

```
{
```

```
    int arr[MAX];
```

```
    int n,k,i,j,found;
```

```
    printf("Enter the size of array : ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter array elements : ");
```



```
for(i=0;i<n;i++)  
{  
    scanf("%d",&arr[i]);  
}
```

```
printf("Enter window size : ");  
scanf("%d",&k);
```

```
printf("Output: ");
```

```
for(i=0;i<=n-k;i++)  
{  
    found=0;
```

```
    for(j=i;j<i+k;j++)  
    {  
        if(arr[j]<0)  
        {  
            printf("%d ",arr[j]);  
            found=1;  
            break;  
        }  
    }  
}
```

```
if(found==0)  
{
```

```

        printf("0 ");
    }
}

printf("\n");

return 0;
}

```

OUTPUT

PS C:\My Files\01_ACADEMICS\SEMESTER_3\DSA\hw_4> ./9

Enter the size of array : 5

Enter array elements : -8 2 3 -6 10

Enter window size : 2

Output: -8 0 -6 -6

=== Code Execution Successful ===

10. Given a matrix of dimension $M \times N$ where each cell in the matrix can have values 0, 1, or 2 which has the following meaning:

0 : Empty cell

1 : Cells having fresh oranges

2 : Cells having rotten oranges

The task is to find the minimum time required so that all oranges become rotten. A rotten orange at index (i,j) can rot a fresh orange which are its neighbours (up, down, left and right). If impossible to rot every orange, return -1. Use Queue for implementation.

```
#include<stdio.h>
```

```
#define MAX 100
```

```
int queue[MAX*MAX][2];
```

```
int front=0,rear=-1;
```

```
void enqueue(int r,int c)
```

```
{
```

```
    rear++;
```

```
    queue[rear][0]=r;
```

```
    queue[rear][1]=c;
```

```
}
```

```
void dequeue(int *r,int *c)
```

```
{
```

```
    *r=queue[front][0];
```

```
    *c=queue[front][1];
```

```
    front++;
```

```
}
```

```
int main()
```

```
{
```

```
    int matrix[MAX][MAX];
```

```
    int m,n,i,j;
```

```
int r,c,nr,nc;
```

```
int fresh=0,time=0;
```

```
int dr[4]={-1,1,0,0};
```

```
int dc[4]={0,0,-1,1};
```

```
printf("Enter rows and columns : ");
```

```
scanf("%d%d",&m,&n);
```

```
printf("Enter matrix elements : ");
```

```
for(i=0;i<m;i++)
```

```
{
```

```
    for(j=0;j<n;j++)
```

```
    {
```

```
        scanf("%d",&matrix[i][j]);
```

```
        if(matrix[i][j]==2)
```

```
        {
```

```
            enqueue(i,j);
```

```
        }
```

```
        if(matrix[i][j]==1)
```

```
        {
```

```
            fresh++;
```

```
    }  
  }  
}
```

```
while(front<=rear && fresh>0)
```

```
{
```

```
    int size=rear-front+1;
```

```
    for(i=0;i<size;i++)
```

```
    {
```

```
        dequeue(&r,&c);
```

```
        for(j=0;j<4;j++)
```

```
        {
```

```
            nr=r+dr[j];
```

```
            nc=c+dc[j];
```

```
            if(nr>=0 && nr<m && nc>=0 && nc<n && matrix[nr][nc]==1)
```

```
            {
```

```
                matrix[nr][nc]=2;
```

```
                fresh--;
```

```
                enqueue(nr,nc);
```

```
            }
```

```
        }
```

```
    }
```

```
        time++;
    }

    if(fresh>0)
    {
        printf("Minimum Time : -1\n");
    }
    else
    {
        printf("Minimum Time : %d\n",time);
    }

    return 0;
}
```

OUTPUT

PS C:\My Files\01_ACADEMICS\SEMESTER_3\DSA\hw_4> ./10

Enter rows and columns : 3 3

Enter matrix elements : 2 1 0 1 1 0 0 1 1

Minimum Time : 4

=== Code Execution Successful ===